SQL Assignment – Medium

Create the database with name as - RetailDB_2

1. Fetch all products along with their supplier name (INNER JOIN).

```
SELECT products.product_name, products.price, suppliers.supplier_name
FROM products INNER JOIN suppliers ON products.supplier_id = suppliers.supplier_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	price	supplier_name
iPhone 15	79999	Reliance Digital
Samsung Galaxy S24	74999	Croma Electronics
Noise Smartwatch	4999	Flipkart Seller Hub
Boat Earbuds	2499	Amazon India Vendor
Kurta Set	1999	Lifestyle Stores
Running Shoes	2999	Lifestyle Stores

2. **Find all customers and their orders, even if they have not placed any (LEFT JOIN).**

```
SELECT customers.name, orders.order_id, orders.total_amount
FROM customers  LEFT JOIN Orders ON customers.customer_id =
orders.customer_id;
```

sult Grid | Filter Rows: | Export: | Wrap Cell Content:

name	order_id	total_amount
Rohit Kumar	31	1499
Rohit Kumar	1	79999
Sneha Sharma	45	74999
Sneha Sharma	26	6499
Sneha Sharma	2	74999
Amit Patel	46	5999

3. Get all suppliers and the products they supply, even if no products exist for a supplier (RIGHT JOIN).

```
SELECT suppliers.supplier_name, suppliers.city,
products.product_name FROM products RIGHT JOIN suppliers ON
products.supplier_id = suppliers.supplier_id;
```

Result Grid | Filter Rows: | Export:

supplier_name	city	product_name
Reliance Digital	Mumbai	iPhone 15
Croma Electronics	Delhi	Lenovo Laptop
Croma Electronics	Delhi	Samsung Galaxy S24
Flipkart Seller Hub	Bengaluru	Noise Smartwatch
Amazon India Vendor	Hyderabad	Boat Earbuds
Tata Cliq	Pune	Prestige Mixer Grinder

Result 10

4. Show all customers and all orders (FULL OUTER JOIN simulation using UNION).

```
SELECT customers.customer_id, customers.name, orders.order_id,
orders.order_date FROM customers LEFT JOIN orders ON
customers.customer_id = orders.customer_id
UNION
SELECT customers.customer_id, customers.name, orders.order_id,
orders.order_date
FROM customers RIGHT JOIN orders ON customers.customer_id =
orders.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	name	order_id	order_date
1	Rohit Kumar	31	2024-03-16
1	Rohit Kumar	1	2024-01-05
2	Sneha Sharma	45	2024-04-13
2	Sneha Sharma	26	2024-03-05
2	Sneha Sharma	2	2024-01-08
3	Amit Patel	46	2024-04-15

Result 11

Output

Action Output

#	Time	Action	Message
21	22:22:07	SELECT customers.name, orders.order_id, orders....	50 row(s) returned
22	22:23:51	SELECT suppliers.supplier_name, suppliers.city, ...	10 row(s) returned
23	22:27:51	SELECT customers.customer_id, customers.name...	50 row(s) returned

5. List all products priced between ₹5000 and ₹50,000 and supplied from "Mumbai".

```
SELECT products.product_name, products.price,
suppliers.supplier_name FROM products JOIN suppliers ON
products.supplier_id = suppliers.supplier_id WHERE
products.price BETWEEN 5000 AND 50000 AND suppliers.city =
'Mumbai';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	price	supplier_name

6. Find the total number of orders placed by each customer and show only those who placed more than 2 (GROUP BY + HAVING).

```
SELECT customers.customer_id, customers.name,
COUNT(Orders.order_id) AS total_orders FROM customers JOIN order
ON customers.customer_id = orders.customer_id GROUP BY
customers.customer_id, customers.name HAVING
COUNT(orders.order_id) > 2;
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	name	total_orders
2	Sneha Sharma	3
3	Amit Patel	3
5	Karan Singh	4
6	Neha Verma	3
7	Arjun Mehta	4
8	Ritika Gupta	4

sult 13

7. Show each supplier's total sales value (sum of quantity × price_each).

```
SELECT suppliers.supplier_id, suppliers.supplier_name,
SUM(Order_Items.quantity * order_Items.price_each) AS
total_sales_value FROM order_Items JOIN products ON
order_Items.product_id = products.product_id JOIN suppliers ON
products.supplier_id = suppliers.supplier_id GROUP BY
suppliers.supplier_id, suppliers.supplier_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

supplier_id	supplier_name	total_sales_value
1	Reliance Digital	399995
2	Croma Electronics	795987
3	Flipkart Seller Hub	30994
4	Amazon India Vendor	17493
7	Lifestyle Stores	40983
5	Tata Cliq	25996

8. Find the average, highest, and lowest price of products in each category.

```
SELECT category, AVG(price) AS average_price, MAX(price) AS
highest_price, MIN(price) AS lowest_price FROM Products GROUP BY
category;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	average_price	highest_price	lowest_price
Electronics	67749.0000	79999	55999
Wearables	3749.0000	4999	2499
Fashion	2499.0000	2999	1999
Home Appliances	6499.0000	6499	6499
Personal Care	1499.0000	1499	1499

9. Find the top 5 customers by total spending (ORDER BY SUM(total_amount) DESC LIMIT 5).

```sql
SELECT customers.customer_id, customers.name,
SUM(orders.total_amount) AS total_spent FROM customers JOIN orders
ON customers.customer_id = orders.customer_id GROUP BY
customers.customer_id, customers.name ORDER BY total_spent DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

customer_id	name	total_spent
8	Ritika Gupta	259996
9	Vikram Joshi	255994
15	Aditya Jain	161997
2	Sneha Sharma	156497
11	Suresh Iyer	139996

10. Show the number of unique products ordered by each customer.

```sql
SELECT customers.customer_id, customers.name,
COUNT(DISTINCT order_Items.product_id) AS unique_products_ordered
FROM customers JOIN orders ON customers.customer_id =
orders.customer_id JOIN order_Items ON orders.order_id =
order_Items.order_id GROUP BY customers.customer_id,
customers.name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	name	unique_products_ordered
1	Rohit Kumar	2
2	Sneha Sharma	2
3	Amit Patel	2
4	Priya Reddy	2
5	Karan Singh	3
6	Neha Verma	2

11. Find customers who placed an order with an amount greater than the average order amount (subquery).

```
SELECT DISTINCT customers.name FROM customers JOIN
orders ON customers.customer_id = orders.customer_id WHERE
orders.total_amount > (SELECT AVG(total_amount) FROM orders);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name
Rohit Kumar
Sneha Sharma
Ritika Gupta
Vikram Joshi
Suresh Iyer
Megha Kapoor

12. Find products that have never been ordered (subquery with NOT IN).

```
SELECT product_name FROM products WHERE product_id NOT
IN (SELECT product_id FROM order_items);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name

13. List customers who ordered at least one product from the "Electronics" category.

```
SELECT DISTINCT c.name FROM customers c JOIN orders o
    ON c.customer_id = o.customer_id
JOIN order_items oi
    ON o.order_id = oi.order_id
JOIN products p
    ON oi.product_id = p.product_id
WHERE p.category = 'Electronics';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name
Rohit Kumar
Sneha Sharma
Karan Singh
Neha Verma
Arjun Mehta
Ritika Gupta

14. Get suppliers who provide products that have been ordered more than 100 times in total.

```
SELECT DISTINCT s.supplier_name FROM suppliers s JOIN
products p ON s.supplier_id = p.supplier_id WHERE p.product_id IN (
SELECT product_id FROM
order_Items GROUP BY product_id HAVING SUM(quantity) > 100 );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

supplier_name

15. Find the most expensive product(s) using a subquery with MAX().

```
SELECT product_name, price FROM products WHERE price = (SELECT
MAX(price)  FROM products);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	price
iPhone 15	79999

16. Show orders placed by customers who live in either Mumbai, Delhi, or Bengaluru (IN operator).

```
SELECT orders.order_id, orders.order_date, customers.name,
customers.city FROM orders JOIN customers ON orders.customer_id =
customers.customer_id WHERE customers.city IN ('Mumbai', 'Delhi',
'Bengaluru');
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	order_date	name	city
1	2024-01-05	Rohit Kumar	Delhi
2	2024-01-08	Sneha Sharma	Mumbai
7	2024-01-22	Arjun Mehta	Bengaluru
13	2024-02-08	Ravi Shankar	Delhi
17	2024-02-16	Arjun Mehta	Bengaluru
23	2024-02-28	Ravi Shankar	Delhi

Result 29

17. Show orders where payment mode is NOT UPI or Credit Card (NOT IN).

```
SELECT * FROM orders WHERE payment_mode NOT IN ('UPI',
'Credit Card');
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	customer_id	order_date	total_amount	payment_mode
3	3	2024-01-10	9998	Debit Card
5	5	2024-01-18	5997	Cash
8	8	2024-01-25	59999	Net Banking
10	10	2024-01-30	2998	Debit Card
12	12	2024-02-05	74999	Net Banking
14	14	2024-02-10	4998	Cash

orders 30

18. Find customers who have no email address recorded (IS NULL).

```
SELECT name FROM customers WHERE email IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name

19. Show suppliers who are not from the same city as any customer (NOT IN subquery).

```
SELECT supplier_name, city FROM suppliers WHERE city NOT
IN (SELECT DISTINCT city FROM customers);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

supplier_name	city

20. Get the latest 3 orders placed, skipping the first 2 (ORDER BY + LIMIT + OFFSET).

```
112 •   SELECT * FROM orders ORDER BY order_date DESC LIMIT 3
113     OFFSET 2;
114
115
116
117
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

order_id	customer_id	order_date	total_amount	payment_mode
48	7	2024-04-19	2999	Cash
47	5	2024-04-17	7999	Credit Card
46	3	2024-04-15	5999	UPI